Итоговые примеры

Поздравляю вас! Еще одна глава позади. Сейчас мы вкратце повторим то, чему научились в этой главе, а затем закрепим пройденный материал на практике.

Оглавление:

1. **Теория**
2. **Задание №1**
3. **Задание №2**
4. **Задание №3**
5. **Задание №4**
6. **Задание №5**

Теория

Всегда используйте круглые скобки для устранения возможных проблем с приоритетами операторов и порядком их выполнения.

Арифметические операторы в языке C++ работают так же, как и в обычной математике. Оператор `%` возвращает остаток от целочисленного деления. Остерегайтесь ошибок округления, когда операнды целочисленного деления и остатка от деления — отрицательны.

Операторы инкремента (`++`) и декремента (`--`) используются для увеличения или уменьшения числа. Остерегайтесь побочных эффектов, особенно когда дело доходит до порядка, в котором будут обрабатываться параметры функции. Не используйте переменную с побочным эффектом больше одного раза в одном стейтменте.

Операторы сравнения позволяют сравнивать числа типа с плавающей точкой. Остерегайтесь использования операторов равенства и неравенства с ними.

Логические операторы позволяют формировать сложные условные стейтменты. Побитовые операторы позволяют работать на уровне отдельных бит.

Задание №1

Вычислите результат следующих выражений:

- `(5 > 3 && 4 < 8)`
- `(4 > 6 && true)`
- `(3 >= 3 || false)`
- `(true || false) ? 4 : 5`

Ответ №1

- `(5 > 3 && 4 < 8) => (true && true)`. Результат: true.
- `(4 > 6 && true) => (false && true)`. Результат: false.
- `(3 >= 3 || false) => (true || false)`. Результат: true.
- `(true || false) ? 4 : 5 => (true ? 4 : 5)`. Результат: 4.

Задание №2

Вычислите результат следующих выражений:

- `7 / 4`
- `14 % 5`

Ответ №2

- `7 / 4 = 1` с остатком `3`. Результат: 1.
- `14 % 5 = 2` с остатком `4`. Результат: 4.

Задание №3

Конвертируйте следующие двоичные числа в десятичную систему счисления:

- 1101
- 101110

Ответ №3

- 1101: `((1 * 8) + (1 * 4) + (0 * 2) + (1 * 1)) = 8 + 4 + 1 = 13`
- 101110: `((1 * 32) + (0 * 16) + (1 * 8) + (1 * 4) + (1 * 2) + (0 * 1)) = 32 + 8 + 4 + 2 = 46`

Задание №4

Конвертируйте следующие десятичные числа в двоичную систему счисления:

- 15
- 53

Ответ №4

Десятичное 15:

Используя метод №1:

```
15 / 2 = 7 r1
7 / 2 = 3 r1
3 / 2 = 1 r1
1 / 2 = 0 r1
```

Смотрим на остатки (снизу вверх): 1111.

Используя метод №2:

```
15 >= 8? Да, 8-й бит равен 1. Остается 7.
7 >= 4? Да, 4-й бит равен 1. Остается 3.
3 >= 2? Да, 2-й бит равен 1. Остается 1.
1 >= 1? Да, 1-й бит равен 1.
```

Результат: 1111.

Десятичное 53:

Используя метод №1:

```
53 / 2 = 26 r1
26 / 2 = 13 r0
13 / 2 = 6 r1
6 / 2 = 3 r0
3 / 2 = 1 r1
1 / 2 = 0 r1
```

Смотрим на остатки (снизу вверх): 110101.

Используя метод №2:

```
53 >= 32? Да, 32-й бит равен 1. Остается 21.
21 >= 16? Да, 16-й бит равен 1. Остается 5.
5 >= 8? Нет, 8-й бит равен 0.
5 >= 4? Да, 4-й бит равен 1. Остается 1.
1 >= 2? Нет, 2-й бит равен 0.
1 >= 1? Да, 1-й бит равен 1.
```

Таким образом, десятичное число 53 равно двоичному 110101.

Задание №5

Почему вы никогда не должны делать следующее:

- `int y = foo(++x, x);`
- `int x = 7 / -2; // (до C++11)`
- `int x = -5 % 2; // (до C++11)`
- `float x = 0.1 + 0.1; if (x == 0.2) return true; else return false;`
- `int x = 3 / 0;`

Ответ №5

- Поскольку оператор `++` создает побочный эффект аргументу `x`, то мы не должны использовать переменную `x` дважды в этом выражении. Параметры функции foo() могут обрабатываться в любом порядке и нельзя определить, что будет первым (`x` или `++x`). Поскольку `++x` изменяет значение `x`, то непонятно, какие значения будут переданы в функцию.
- До C++11 непонятно, округлит ли компилятор это значение до `-3` или до `-4`.
- До C++11 непонятно, будет ли результатом `1` или `-1`.
- Ошибки округления со значениями типа с плавающей точкой приведут к результату `false`, хоть и кажется, что должно быть `true`.
- Деление на 0 приведет к сбою в программе.